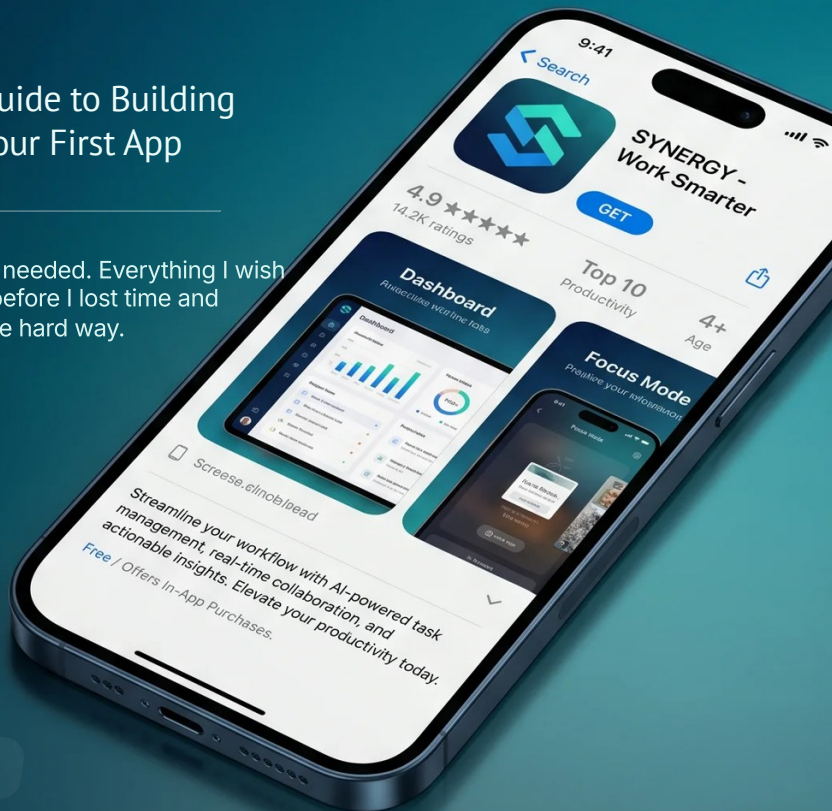


A NO-NONSENSE LAUNCH GUIDE

# From Idea to App Store

The Beginner's Guide to Building and Launching Your First App

No coding background needed. Everything I wish someone had told me before I lost time and money figuring it out the hard way.



# Contents

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>How to use this guide</b>                                        | <b>3</b>  |
| <b>1</b> Finding an App Idea That's Actually Worth Building         | <b>4</b>  |
| <b>2</b> What This Is Actually Going to Cost You                    | <b>9</b>  |
| <b>3</b> App Builders to Avoid (and Why)                            | <b>12</b> |
| <b>4</b> My Preferred App Builder and How It Works                  | <b>15</b> |
| <b>5</b> Setting Up Your Accounts (Start These Early!)              | <b>21</b> |
| <b>6</b> Building Your App, FlutterFlow + Firebase Tips and Tricks  | <b>26</b> |
| <b>7</b> How Your App Makes Money                                   | <b>31</b> |
| <b>8</b> Building a Simple Landing Page (with Free Hosting)         | <b>34</b> |
| <b>9</b> Privacy Policy and Terms of Service (the Painless Version) | <b>37</b> |
| <b>10</b> Creating Great Visuals to Promote Your App                | <b>40</b> |
| <b>11</b> Beta Testing Before You Launch                            | <b>43</b> |
| <b>12</b> Submitting to the App Store and Google Play               | <b>46</b> |
| <b>13</b> You're Live. Now What? (Post-Launch)                      | <b>50</b> |
| <b>A final word</b>                                                 | <b>53</b> |

## Before you start: how to use this guide

I wrote this for one specific person: you, a few months ago, before you knew any of this. Someone with a good idea, zero technical background, and no clear sense of what the actual steps are or what they cost.

Here's my promise: if you follow this guide in order and do the action step at the end of each chapter, you'll go from "I have an idea" to "my app is live" without getting lost, and without the expensive surprises that catch most first-timers.

A few ground rules:

- **Go in order.** Each chapter sets up the next. The boring early chapters (costs, accounts) exist because skipping them is exactly what costs people time and money.
- **Do the action step before moving on.** Reading isn't building. The action steps are what get your app made.
- **Don't panic at the scary words.** DUNS numbers, signing certificates, app review, every intimidating term in here gets explained in plain English. None of it is as hard as it sounds.

Let's build something.

# Finding an App Idea That's Actually Worth Building

---

When I started, I thought the hardest part of building an app would be the building. I was wrong. The hardest part, and the part that quietly wastes the most time and money for first-timers, is picking the *right idea* before you've spent a dollar or an hour on it.

I learned that the expensive way. So this chapter is the one I wish someone had handed me on day one. By the end of it you'll have a real idea, you'll have pressure-tested it, and you'll know whether it's worth taking to the next chapter. No code, no accounts, no spending. Just thinking clearly before you commit.

Take your time here. An afternoon spent on this chapter can save you weeks later.

## Where good app ideas actually come from

Forget the myth of the lightning-bolt idea that nobody's ever thought of. The best beginner apps almost never come from "inventing something brand new." They come from one of three much more reliable places:

**1. A problem you personally have.** This is the strongest starting point because you already understand the user, you are the user. You know what's annoying, what you'd pay to fix, and what "good" looks like. If you've ever thought "why isn't there just an app that does X," write that down. That frustration is worth more than a clever idea you don't actually care about.

**2. A small, specific group with a specific need.** Big, broad apps ("a social network," "a better email") are where beginners go to drown. Narrow is your friend. "An app for dog walkers to log routes" beats "an app for pet owners." "A tip-splitting app for restaurant servers" beats "a finance app." The narrower the audience, the easier it is to build something they actually love, and the easier it is to find them later when you want to market it.

**3. An existing app that frustrates people.** You don't need a new idea. You need a *better* version of something people already use and complain about. This is one of the most underrated strategies, and we'll use it as a research tool in a minute.

**A quick gut-check:** The best first app is small enough that you could describe everything it does in one sentence, and useful enough that one specific person would be a little sad if it disappeared. If your idea needs a paragraph to explain, it's probably too big for a first build. Shrink it.

## The 20-minute brainstorm

Don't try to find your idea in your head. Get it on paper. Here's a brainstorm that actually produces something usable:

1. **List 10 things that annoyed you this week.** Anything, forgetting to water plants, splitting a bill, tracking a habit, finding a quiet café. Don't filter. Bad ideas unlock good ones.
2. **List 3 things you know more about than the average person.** A hobby, a job, a skill. Apps built on knowledge you already have are dramatically easier to design well.
3. **For each item, finish this sentence:** "There should be an app that just lets me \_\_\_\_."

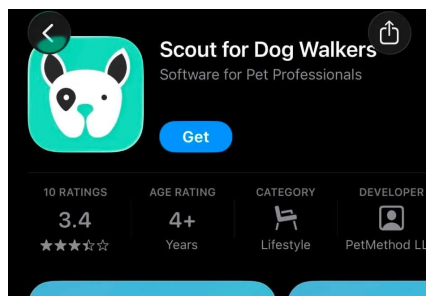
Circle the two or three that make you a little excited. Excitement matters more than you'd think, you're about to spend real time on this, and the boring "smart" idea is the one you'll abandon in week three.

## Now validate it, this is the step I skipped

Here's where I lost time and money the first time around: I fell in love with an idea and started *building* before I checked whether it made any sense. Don't do that. Validation is just answering four questions honestly, and it costs you nothing but an hour of searching.

### Question 1: Does this already exist?

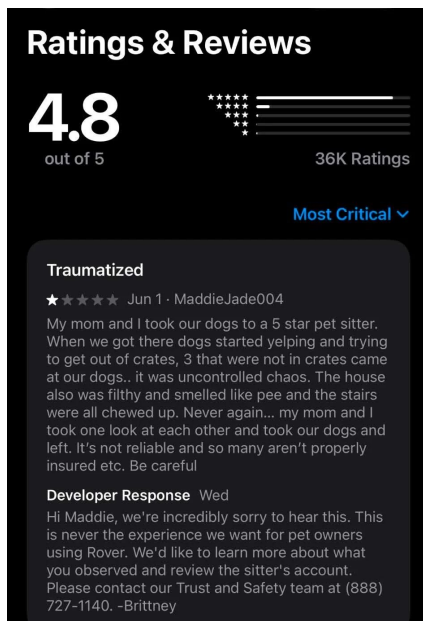
Search the App Store and Google Play for your idea. Search Google too. You're not hoping for empty results, empty results are usually a *bad* sign (it often means there's no demand, not that you found a gold mine). What you want to find is a few existing apps that are doing okay but aren't great. Example:



## Question 2: What do people hate about what already exists?

This is the single most valuable research trick in this whole guide, so slow down here. **Go read the 1-star and 2-star reviews of your competitors.** Not the 5-star ones, the angry ones. People will tell you, for free and in detail, exactly what's broken, missing, or frustrating. That list of complaints is your feature list. You're not copying the competitor; you're building the version that fixes what people are already begging to have fixed.

Open a note and write down every complaint that comes up more than once. Patterns are your opportunity. Example:



## Question 3: Is anyone actually looking for this?

You want signs of real demand. A few quick, free ways to check: - Type your idea into Google and watch the autocomplete suggestions, those are real things people search for. - Look at whether competitor apps have lots of reviews (lots of reviews = lots of users = real market). - Search Reddit or relevant forums for people describing the problem in their own words. If people are complaining about the problem online, that's demand.

You're not running a science experiment. You're just looking for evidence that you're not the only person on earth who wants this.

#### Question 4: Can a small version of this actually help someone?

Be brutally honest. Strip your idea down to the *one thing* it must do to be useful. Everything else is a "later" feature. If the stripped-down version still solves a real problem for a real person, you've got something. If it only works once you've built fifteen features, it's too big, go back and shrink it.

**The "version 1" rule:** Your first app should do one thing well, not ten things adequately. You can always add more after launch. You cannot recover the months you'll lose trying to build everything at once.

### Red flags that should make you pause

Before you commit, check your idea against these. Any one of them isn't necessarily a dealbreaker, but two or more is a sign to rethink:

- **It needs a huge number of users to be useful at all.** (Anything where the whole point is that "everyone" is on it, these are brutally hard for a first app.)
- **It depends on data or content you can't get.** (E.g., needs a constantly updated database you don't have access to.)
- **It needs to handle people's money or sensitive health/legal data on day one.** (Adds serious legal and technical weight, fine later, heavy for a first build.)
- **You're not actually interested in the problem.** (You'll quit. Be honest.)
- **You can't explain it in one sentence.** (Too big. Shrink it.)

### Your action step: the one-page idea validation worksheet

Don't move to the next chapter until you've filled this out. Copy these prompts into a note and answer them. If you can answer all of them with confidence, you have a validated idea and you're genuinely ready to build.

**My app in one sentence:**

"My app lets [who] do [what] so they can [benefit]."

**The one core thing it must do (version 1):**

**Who exactly is this for (be specific):**



**Competitors I found:**



- 1.
- 2.
- 3.

**Top 3 complaints from their bad reviews (my opportunity):**



- 1.
- 2.
- 3.

**Evidence that people actually want this:**



**Red flags I need to watch:**



If a few of those blanks are hard to fill, that's not failure, that's the worksheet doing its job. Better to find the weak spot now, on paper, than three months and a few hundred dollars from now.

When this page is filled in and it still feels exciting? Good. That's your green light. In the next chapter we'll talk about exactly what this is going to cost you, all of it, no surprises, so you can decide with your eyes open before you spend a cent.



# What This Is Actually Going to Cost You

---

Nobody told me the real number before I started, and the surprises came one at a time, a fee here, a subscription there, a "wait, I need to pay for *that* too?" each time I hit a new step. Drip by drip, it added up to way more than I'd budgeted, and every surprise made me question whether I should keep going.

So this chapter does the thing nobody did for me: it lays out every cost up front, honestly, so there are zero surprises. You'll know the real all-in number before you spend a single dollar. Some of these are unavoidable, some are optional, and I'll tell you which is which.

## The unavoidable costs

These are the ones you genuinely cannot skip if you want to publish to the app stores.

**Apple Developer Program, \$99 per year.** This is what lets you put apps on the iPhone App Store. It's a yearly fee, not one-time, so budget for it every year you keep your app live. Same price whether you sign up as yourself or as a business.

**Google Play Developer account, \$25, one time.** This is the Android equivalent, and it's a much friendlier deal: you pay \$25 once, ever, and you can publish unlimited apps forever. No annual renewal.

**A domain name, roughly \$10-15 per year.** You need a website address (like [myapp.com](https://myapp.com)) for two reasons: it's where your landing page lives, and, this is the part that surprises people, both Apple and Google *require* a publicly visible privacy policy at a real web address before they'll approve your app. We'll cover all of this later; for now, just know it's a small yearly cost you can't skip.

**You can launch on just one store to start.** If money's tight, nothing says you have to launch on both Apple and Google on day one. Many people start with one. Android is the cheaper entry point at \$25 one-time; iPhone reaches an audience that tends to spend more. We'll come back to this decision.

## The "it depends" costs

These depend on how you build, and they're where the monthly numbers come from.

**Your app builder (FlutterFlow), about \$39/month to actually publish.** I'll explain FlutterFlow in detail in Chapter 4. The important budgeting fact right now: it has a free plan, but **the free plan won't let you publish to the app stores or export your work.** To actually ship an app, you need a paid plan, which starts around \$39/month. You only need to pay during the months you're actively building and updating, which is worth knowing, more on that trick in Chapter 4.

**Your backend (Firebase), free to start, scales with users.** Firebase is the service that stores your app's data (user accounts, saved info, etc.). The good news: it's free until you have a meaningful number of users, so as a beginner launching, you'll almost certainly pay nothing here at first. It only starts costing money once your app is popular enough to need it, which is a good problem to have. We'll set it up in Chapter 6.

## The optional costs (skip these at the start)

- **An app icon / graphics designer.** You can absolutely make your own visuals for free at the start (Chapter 10 shows you how). Hire a designer later if you want to.
- **Paid legal templates.** There are free, legitimate ways to generate a privacy policy and terms (Chapter 9). You don't need a lawyer for a simple first app.
- **Ads / marketing budget.** Spending *money* on ads is optional, especially at the start. But don't file marketing under "later", have a simple, free plan ready *before* you launch: where you'll share it, which communities your users are in, who you'll tell first. The budget is optional; the plan isn't. (Chapter 13 walks you through building that plan, no ad spend required.)

## The cost nobody puts on the list: your time

This is real, so I'll say it plainly. Your first app will take longer than you expect, not because it's hard, but because you're learning. Budget yourself a few unhurried weeks, not a frantic weekend. The people who quit are almost always the ones who expected it to be instant. You won't make that mistake, because I just told you.

## Your action step: fill in your real budget

Copy this and fill in your own numbers. Seeing the actual total, and realizing most of it is small or one-time, is what makes people feel safe to start.

| Cost                               | Amount         | When                      | Skippable?           |
|------------------------------------|----------------|---------------------------|----------------------|
| Apple Developer Program            | \$99           | Per year                  | Only if Android-only |
| Google Play account                | \$25           | One time                  | Only if iPhone-only  |
| Domain name                        | ~\$12          | Per year                  | No                   |
| FlutterFlow (paid plan)            | ~\$39          | Per month, while building | No (to publish)      |
| Firebase                           | \$0 to start   | When you grow             | Free at first        |
| Business email (Google Workspace)  | ~\$7/user      | Per month                 | Only if organization |
| Designer / extras                  | \$0            | Optional                  | Yes                  |
| <b>My realistic starting total</b> | <b>\$_____</b> |                           |                      |

For most people launching on both stores, the real "get started" number lands around **\$150-200 to begin**, most of which is the first month of FlutterFlow plus the two developer accounts. That's it. That's the wall that stops people, and now it's not a surprise anymore. (If you register as an *organization* rather than an individual, add roughly **\$7/month** for a business email, see Chapter 5, and note that's the only extra; the DUNS number itself is free.)

Next up: the app builders I'd steer you *away* from, and exactly why, so you don't make the mistake of starting on the wrong tool.

# App Builders to Avoid (and Why)

---

When you search "how to build an app without coding," you get buried in options, and they all claim to be the easy answer. Some are great. Some will waste your money, trap your work, or quietly fail you right at the finish line when you try to publish. I tried some of the wrong ones so you don't have to.

This chapter isn't me trashing specific companies, tools change, prices change, and what's bad today might be good next year. Instead I'm going to teach you the *red flags*, so you can judge any builder yourself, including ones that don't exist yet.

## Red flag 1: You can't get your app into the actual app stores

This is the big one. A shocking number of "app builders" only make a *web app*, a website that looks like an app, or trap your creation inside their own viewer app. When you go to put your thing on the real App Store and Google Play, you discover you can't, or you can only through some awkward, limited workaround.

**How to check before you commit:** search "[builder name] publish to App Store" and "[builder name] publish to Google Play." If you can't find clear, recent confirmation that real apps get published to both stores, walk away.

**Watch the "wrapper" trap especially, I fell straight into it.** This one hits people who use the trendy *prompt-based* builders, where you describe your app in plain English and AI generates it. The prompts themselves cost money (you buy credits), and "just one more tweak" burns through them faster than you'd believe. But here's what actually got me: I *finished* building my app on one of these, only to find out that what it produced was basically a web app that couldn't go into the stores on its own. To publish it, I had to "wrap" it with a separate service, in my case Median.co, which meant paying **\$229 for the license and then \$179 a year for the starter pack**, on top of everything I'd already spent on prompts. So I paid to build it, then paid again just to make it publishable.

And that wasn't even the worst part. **These prompt-based builders tend to come with a very limited backend**, the backend being the part that stores your users, their accounts, and their data (you'll meet this as "Firebase" in Chapter 6). That limit doesn't bite at first. It bites the moment you start to *succeed*: I hit a small number of subscribers and ran straight into the

ceiling, with no real way to grow. My only option was to rebuild the entire app somewhere else and start over, losing the time and money I'd already sunk into the first version. Two traps, one expensive lesson.

This is a big reason I recommend the builder in Chapter 4: it produces real, native apps that go straight to both stores with **no wrapper and no extra subscription**, and it connects to a proper backend that grows with you instead of capping you the moment things go well. If a tool's apps need wrapping to be publishable, or its backend can't handle more than a handful of users, count both of those as part of its true price.

## Red flag 2: They hold your work hostage

Some platforms never let you export what you've built. If you ever want to leave, because they raised prices, shut down, or just aren't working for you, you lose everything and start over. You're renting, not owning.

**What you want instead:** a builder that lets you *export your code* or otherwise take your work with you. (This is one of the specific reasons I landed on the builder I recommend in the next chapter, your work is yours.)

## Red flag 3: Surprise per-app or per-user fees

Watch the pricing page like a hawk. Some tools advertise a low monthly price, then charge extra *per app you publish*, or *per user who signs up*, or to "remove our branding" from your app. A cheap-looking plan can quietly become expensive the moment you actually succeed.

**How to check:** find the pricing page and read it slowly. Look specifically for the words "per app," "per seat," "per user," "branding," and "add-on." If the real cost only appears once you're successful, that's a trap.

## Red flag 4: A ghost town around it

When you're stuck, and you will get stuck, you need help. The best builders have huge communities: YouTube tutorials, active forums, Facebook groups, official docs. A tool with no community means that every time you hit a problem, you're on your own.

**How to check:** search YouTube for "[builder name] tutorial." A healthy tool has hundreds of recent videos from real users. A risky one has a handful, all from the company itself.

**The quick test for any builder:** Can I publish to both app stores? Can I take my work with me if I leave? Is the *real* price (after add-ons) something I can afford? Is it actually made for beginners? Is there a big community to help me when I'm stuck? Five yeses, or keep looking.

## A gentle word on the "AI will build my whole app" tools

You'll see a wave of tools promising that AI builds your entire app from a sentence. Some are genuinely useful and I'll touch on where AI helps in later chapters. But as a beginner betting real time and money, be cautious: many produce something that demos beautifully and then falls apart the moment you need it to do something specific, publish to a store, or handle real users. Treat "describe it and we'll build it" claims with healthy skepticism until you've seen real, published apps come out the other end.

## Your action step: run the test on your shortlist

Before the next chapter, take any 2-3 builders you've been curious about and run them through the five-question test above. Write down a yes/no for each question. This does two things: it shows you *why* the tool I recommend next is the one I trust, and it trains your instincts so you never get talked into the wrong tool by a slick sales page.

In the next chapter I'll introduce the builder I actually use and recommend, walk you through what it is, how it works, and the honest truth about its free-versus-paid plans.

# My Preferred App Builder and How It Works

---

After trying the wrong tools, I landed on **FlutterFlow**, and it's what I'd put in the hands of anyone starting today. This chapter is the friendly tour: what it is, why I trust it, how it actually works, and the honest truth about its pricing so you're never surprised.

## What FlutterFlow actually is

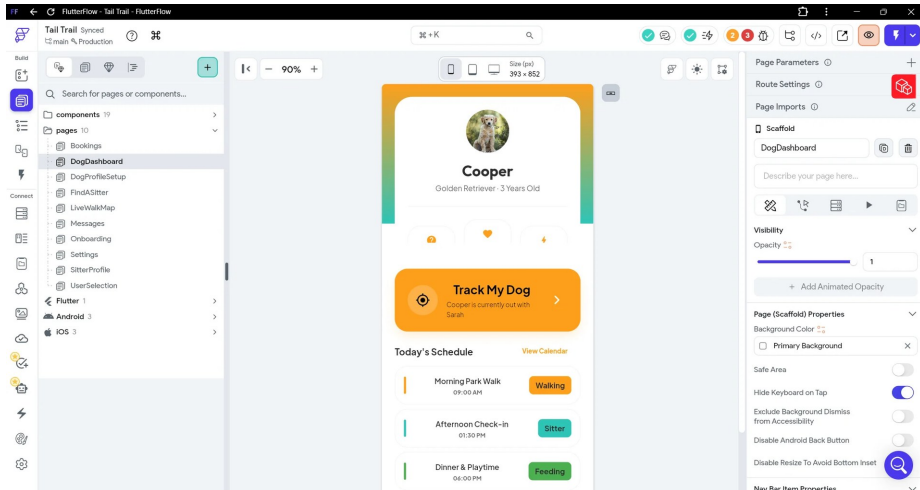
FlutterFlow is a **visual** app builder. Instead of typing a prompt and having an AI spit out a finished app for you, the prompt-based approach that landed me in the traps from the last chapter, *you* build the app yourself, piece by piece. You drag and drop the parts, buttons, text, images, screens, onto a canvas, the way you'd arrange things in a slideshow or a design tool, and you watch your app take shape as you go. Because you're the one building it, you actually understand it, you own it, and what comes out is a real native app, no wrapper, no forced rebuild later.

Now I want to be straight with you, because the cover promised I would: FlutterFlow is *mostly* drag-and-drop, but it isn't *purely* drag-and-drop. Every so often you'll hit something that needs a small piece of custom code, a custom function, a bit of logic the visual tools don't cover on their own. If you've never written a line of code, that probably sounds intimidating. It shouldn't, and here's why: **you don't have to write that code yourself.**

When I need a custom function or a snippet for FlutterFlow, my go-to is **Claude (the AI assistant from Anthropic)**. I describe what I want in plain English, "write me a function that does X", Claude writes the code, and I paste it straight into FlutterFlow. It'll also explain what the code does and help me fix it if something breaks. So even the "coding" parts of FlutterFlow don't actually require you to know how to code: you describe, it writes, you paste. (And to be clear, this is *very* different from the prompt-builder trap, there, the AI builds and owns your whole app on a platform you can't leave. Here, the AI just hands you a small piece that goes into an app you fully control.)

And custom code is only the *occasional* part. The everyday work, what you'll actually spend most of your build time on, is the **wiring**: you're the one adding the actions and building the simple logic that makes the app behave. "When this button is tapped, save the entry and go to

the next screen", you set that up yourself, step by step, piece by piece. Nobody does this part for you, and that's exactly why the finished app ends up genuinely *yours*. It's also far easier than it sounds. Whenever I'm wiring up a screen and I'm not sure how to set something up, I work through it with Claude one step at a time, I tell it what I'm trying to make happen, and it walks me through which actions to add and in what order. Done that way, the wiring goes from the scariest-sounding part to honestly one of the most satisfying. We'll see all of this in action in Chapter 6.



Here's why it earns my recommendation against the red flags from the last chapter:

- **It publishes to both real app stores.** Apple App Store and Google Play. The whole point.
- **Your work is yours.** On a paid plan you can export your actual app code and take it with you. You're not trapped.
- **It's built on serious technology.** Under the hood it produces apps made with "Flutter," the same framework used by huge professional apps. You're not building a toy.
- **The community is enormous.** YouTube tutorials, an active forum, docs, courses. When you get stuck, the answer almost always already exists.

## Set Claude up the right way first: one Project just for your app

I've mentioned leaning on Claude a lot, so let me show you the single setup decision that makes it far more useful: **create a dedicated Project in Claude for this app, and do all your app-related chatting inside it.**

Here's why it matters. In Claude, a "Project" is a workspace that keeps a group of chats together, and each Project has its own **memory**. That means Claude builds up and holds on to context about *your specific app* across all the conversations inside that project: what your app does, how you've set things up, decisions you've made, problems you've already solved. Instead of re-explaining your app from scratch every time, Claude already knows it. (Outside of a project you don't get that app-specific continuity, which is exactly why this is worth doing before anything else.)



A practical reality you'll hit: a single chat can't go on forever. As you paste in screenshot after screenshot over days of building, one conversation eventually gets full and starts to slow down or stop accepting more, and you'll need to start a fresh chat. That's completely normal. Because all your chats live in the same project, a new one still shares the project's memory, but here's the trick that makes the switch seamless:

**Before you start a new chat, ask Claude for a handoff.** Say something like: *"We're moving to a new chat. Write me a prompt I can paste into the next one that captures where we're at, what we've done so far, and what still needs doing, keeping the same flow and structure we've been using."* Claude hands you a tidy summary; you paste that in as the first message of the new chat, and you pick up exactly where you left off with no momentum lost. Do this every time a chat fills up, and your whole build stays one continuous thread across as many chats as it takes.

This habit, one project, plus a handoff prompt whenever a chat fills up, is what turns Claude from a forgetful helper into something that genuinely feels like it's building the app alongside you the whole way.

One honest thing before we go further, though, because it matters more than anything else in this chapter: **Claude is a tool, not autopilot.** Lean on it heavily, yes, but stay awake while you do. Every time Claude walks you through a step, actually pay attention and *learn* what's happening, so the next time you can do it yourself (or catch it when something's off). If you just zone out and copy-paste whatever Claude gives you without understanding it, I promise you'll end up in some very unpleasant rabbit holes down the line, trust me on that one. Use your own logic. Challenge Claude when something doesn't make sense, or when you suspect there's a simpler way to solve your problem, there often is, and a bit of push-back gets you a better answer. Don't be afraid to say "wait, that doesn't seem right" or "isn't there an easier way to do this?" The people who build great apps with Claude are the ones treating it like a sharp assistant they're working *with*, not a vending machine they're mindlessly copying from. Learn as you go, and Claude becomes incredibly powerful. Switch your brain off and just paste... well, good luck.

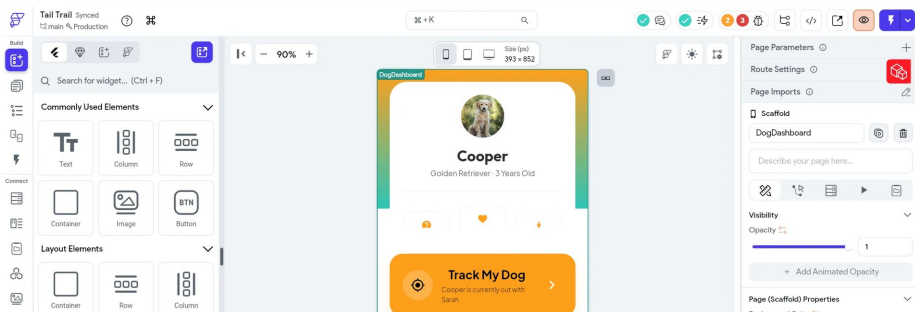
## How it works, in plain terms

Building an app in FlutterFlow comes down to three things, and once they click, the whole tool makes sense:

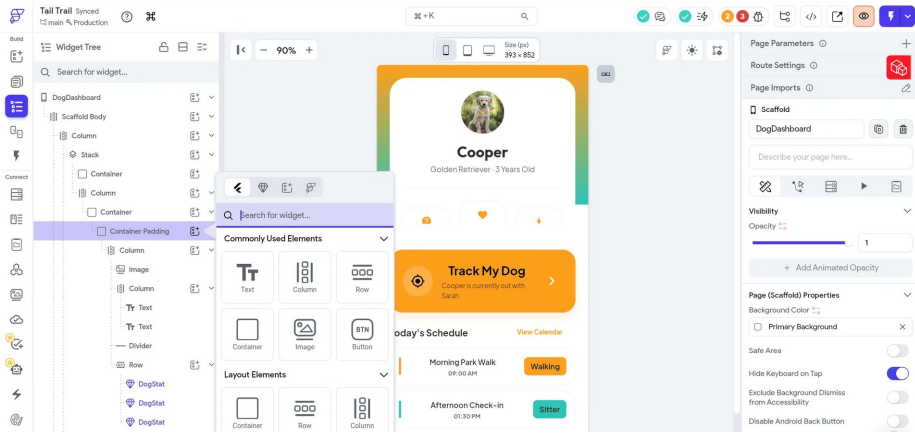
**1. Screens (called "pages").** Every app is just a set of screens the user moves between, a login screen, a home screen, a settings screen. In FlutterFlow you create each screen on its own canvas.

**2. Widgets (the pieces on each screen).** Everything you see in an app, a button, a line of text, an image, a list, is a "widget." You drag widgets onto your screen and arrange them. That's the bulk of the design work, and it's genuinely point-and-click. You can also add widgets directly from the widget tree. See examples below:

From the widget palette  
(drag & drop) →



From the widget tree  
(click + and select) →



**3. Actions (what happens when you tap).** This is the part that makes it an app and not a picture. You tell a widget what to *do*: "when this button is tapped, go to the next screen" or "save this text." You set these up by picking from menus, no typing code.

That's the core loop: build a screen, place your widgets, tell them what to do when tapped. Repeat for each screen, connect them together, and you have an app.

## Where Firebase fits in

You'll hear me mention **Firestore** a lot, so here's the one-sentence version: FlutterFlow builds what your app *looks like and does*, and Firebase is the free service that *remembers things*, user accounts, saved data, anything that needs to stick around. The two are designed to work together, and FlutterFlow connects to Firebase with a guided setup. We'll do that together in Chapter 6. For now, just hold the mental model: **FlutterFlow = the app, Firebase = the memory.**

## The honest truth about pricing

I won't sugarcoat this, because the free plan tripped me up at first.

**The free plan is for learning, not launching.** It's genuinely great for poking around, watching your idea come to life, and getting comfortable. But it will *not* let you export your code or publish to the app stores. If you only ever use the free plan, you can build something on screen but you can never actually ship it.

**To publish, you need a paid plan,** the entry-level paid tier starts around **\$39/month**. That unlocks publishing to the stores and the ability to download your work.

**A money-saving trick:** FlutterFlow is billed monthly, and you only truly need the paid plan during the stretches when you're actively building and publishing. Some people build on the free plan to learn, then upgrade for the month or two when they're finishing and shipping, then downgrade. You don't have to pay year-round if you're not actively working on the app. (Just don't cancel mid-publish.)

## Your action step: draft your first build prompt with Claude

You've got your idea (Chapter 1) and you know your tools (this chapter). So your action step here isn't to start building, it's to get the *prompt* ready that you'll later feed into FlutterFlow's AI builder to generate your app's base (the Step 2 move from Chapter 6). And you don't write that prompt alone, you have Claude write it for you.

Here's how it works. There are actually two prompts at play, so don't mix them up: 1. **The brief you give Claude**, a plain-English description of your app (you fill in the template below). 2. **The prompt Claude hands back**, a polished, detailed version, written the way an app builder likes it, that you'll paste into FlutterFlow later.

So: open your app's Claude project (the one you just set up), grab your Chapter 1 validation worksheet, and send Claude something like this:

### Copy/paste template, the brief you send Claude:

*"I'm building my first mobile app using FlutterFlow, and I want to use FlutterFlow's AI builder to generate a starting base. Can you write me a clear, detailed prompt I can paste into FlutterFlow's AI page/app generator?"*

*Here's my app: - What it does (one sentence): \_\_\_\_\_ - Who it's for: \_\_\_\_\_ - The main thing a user needs to be able to do: \_\_\_\_\_ - The screens I think it needs: \_\_\_\_\_ (e.g. login, home, [your core screen], settings) - Any data it needs to store: \_\_\_\_\_ (e.g. user accounts, saved entries) - The general look/vibe I want: \_\_\_\_\_ (e.g. clean and minimal, bright and playful)*

*Please turn this into a single, well-structured prompt for FlutterFlow's AI builder. Ask me anything you need before writing it."*

Claude will take your rough answers and hand you back a tight, detailed prompt, and because you're in your app's project, it'll remember all of this for every chat that follows. Save that

prompt somewhere handy. When you get to the build in Chapter 6, that's the exact thing you'll paste into FlutterFlow to generate your base.

That's the goal of this step: not a finished app, but a clear, ready-to-go plan in prompt form, and proof that you and Claude are already building this thing together.

Next, before we build for real, we'll set up the boring-but-critical accounts, Apple, Google, your domain, because some of them take days to come through, and starting them now means you're not stuck waiting later.

# Setting Up Your Accounts (Start These Early!)

---

This is the least glamorous chapter in the guide, and also one of the most important, because some of these steps take *days* to process, and if you wait until your app is finished to start them, you'll be sitting around unable to launch, watching a "pending verification" screen. So we do them now, early, in the background, while you build.

Think of this chapter as starting a few slow-cooking pots before you begin the rest of the meal.

## First, one decision: are you an "individual" or an "organization"?

Both Apple and Google ask you this the moment you sign up, and it matters more than it looks, especially on Google, where **you can't change your mind later**. So let's get it right the first time.

**Individual (or "personal").** You're signing up as yourself. Your own legal name shows up as the app's seller. It's simpler, faster, needs no DUNS number, and is perfect for most people launching a first app.

**Organization (or "business").** You're signing up as a registered company. Your business name shows as the seller. This requires a **DUNS number** (explained below) and takes longer, because the platform verifies your business. Choose this if you have a real registered business and want to launch under its name.

**The DUNS number, demystified, and the part everyone gets wrong.** You may have heard you "need a DUNS number to publish an app." Here's the truth that saves a headache: **the DUNS number follows your account type, not the store.** You only need one if you enroll as an *organization*, on *either* Apple or Google. Enroll as an *individual* and you skip it entirely on both. (A DUNS number is just a free nine-digit ID that a company called Dun & Bradstreet assigns to businesses to confirm they're real.) This trips people up constantly: you might breeze through one store and then get stuck waiting on a DUNS at the other, and assume the two stores have different rules. They don't. It's almost always because you picked *individual* on one and *organization* on the other. Pick the same type on both and you'll need a DUNS for both or neither. And if you do go organization, request the DUNS the *very first thing* you do, it's free, but it can take anywhere from a few days to a couple of weeks, and nothing else proceeds without it.

## The limitations of going "individual" (read this before you choose)

Individual is the easy path, but it comes with real tradeoffs. None are dealbreakers for most first apps, but you want to choose *knowing* them, not stumble into them later:

- **Your personal name is public.** On an individual account, *your* legal name (not a business name) shows as the developer on the store listing. If you'd rather not have your own name out there for anyone to see, that alone may push you toward an organization account.
- **On Google, you're locked into the 12-tester rule.** Individual/personal accounts must complete the closed test, around 12 testers over 14 days, before you're allowed to publish. Some organization accounts are exempt from this (though Google keeps expanding the requirement). We cover exactly how to do it in Chapter 11.
- **On Google, you can't switch later, and this is the big one.** Once you create a *personal* Google Play account, you cannot convert it to an organization account, or the other way around. To change, you'd have to create a brand-new account, pay the \$25 again, and your existing apps *can't be transferred over*. So "individual" on Google isn't a temporary shortcut you upgrade from later, it's a fork in the road. Choose it deliberately.
- **Some app types require an organization, period.** If your app deals with financial services, banking, loans, stock trading, investment funds, crypto wallets or exchanges, Google requires an organization account. Individual isn't even an option for those.
- **Apple is friendlier here.** Unlike Google, Apple *does* let you upgrade an individual account to an organization later. So on the Apple side, starting as an individual is far less of a one-way door.

The upshot: apart from the points above, individual and organization accounts have the same features, and both can make money. So the real questions are just: Do I want my own name on it? Am I fine with the 12-tester step? Might I want a business account down the line? If you're a solo maker launching a personal project and you're okay with your name on it, individual is great and saves you weeks. If there's a real business behind this, or you'll want the business name shown, go organization from the start, especially on Google, since you can't switch afterward.

**Watch the signup screen itself, it's sneaky.** When you start your Google Play account, it asks you to "choose an account type" and shows two options: **An organization** and **Yourself** (the individual one). Heads-up: if you click into the organization dropdown, it can expand and *hide the "Yourself" option sitting right below it*, making it look like individual isn't even offered. It is. Just close the dropdown or scroll down and you'll find it. Don't let a covered-up menu trick you into the wrong, unchangeable choice.



## To get started, choose an account type

Who are you creating an account for? [Learn more about which account type to choose](#)

### An organization

Choose if you're creating an account for an organization or business. You'll need to verify your organization.

Organization type —

[Get started](#) →



### Yourself

Choose if you're creating an account for yourself, and you don't currently have an organization or business. For example, if you're an amateur developer, student, or hobbyist. You'll still be able to earn money on Google Play, and invite others to join your account.

ⓘ Some types of apps can only be distributed by organizations.  
[Learn more](#)

[Get started](#) →



So before anything else: decide individual or organization, *on purpose*. Then start the accounts below.

## If you registered as an organization: set up a business email

Here's something that catches organization registrants off guard, you'll be expected to use a **professional business email on your own domain** (something like `you@yourbusiness.com`), not a personal Gmail address. It makes your business look legitimate and it's what's expected when you verify an organization account. The good news: you already bought a domain earlier in this chapter, so you can put an email right on it. The most common way is **Google Workspace** (the paid, business version of Gmail). Here's the quick version:

1. Go to the Google Workspace website and click **Get Started**.
2. Enter your business name and details, and the domain you bought.
3. **Verify that you own the domain.** Google walks you through this, it usually just means adding a small verification record at your domain registrar (the place you bought the domain), and Google gives you the exact thing to paste in.

4. Create your email address, e.g. `you@yourdomain.com`, and a password.
5. Pick a plan. The entry-level **Business Starter** plan runs about **\$7 per user per month** on annual billing, and one user is all you need to start.
6. Google will have you **update your domain's "MX records"** (the setting that routes email to Google). It gives you the exact values and a step-by-step guide; you paste them in at your registrar, and within a little while your email is live.

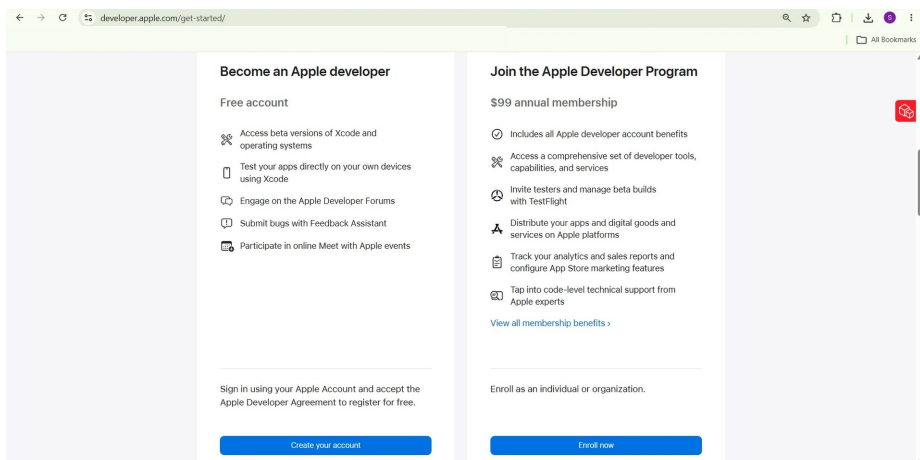
That's it, a professional inbox at your own domain, ready for your developer accounts and anywhere else your business appears. (There are cheaper email-only alternatives if you prefer, but Workspace is the most common and best-documented, and as always, Claude can walk you through any step that trips you up.)

If you went the **individual** route, you can skip all of this, your personal email is fine.

## Step 1: Apply for your Apple Developer account

This is your ticket to the iPhone App Store.

1. Go to the Apple Developer site and sign in with an Apple ID (make one if needed, consider a dedicated one just for your app business).
2. Turn on two-factor authentication if it isn't already (Apple requires it).
3. Enroll in the **Apple Developer Program**. Choose individual or organization per your decision above.
4. Pay the **\$99 annual fee**.



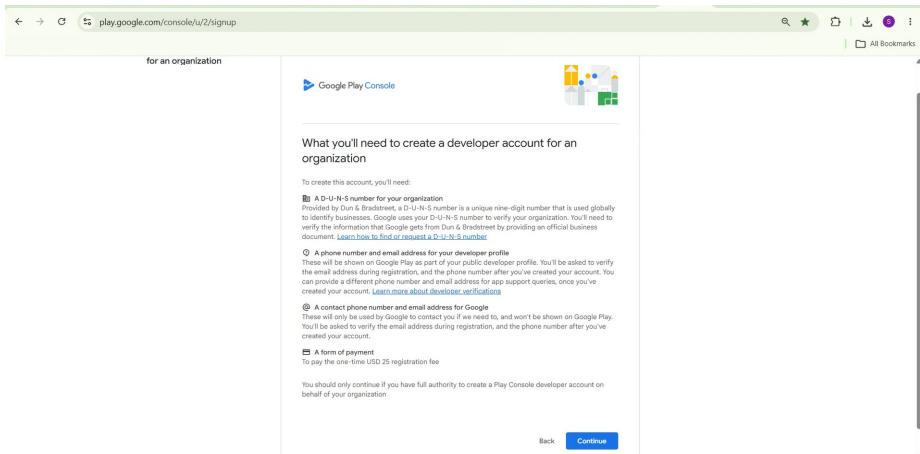
**Lead time:** Individual enrollment is often approved quickly, sometimes within a day. Organization enrollment takes longer because Apple verifies your business (and your DUNS number). This is exactly why we start now.



## Step 2: Apply for your Google Play Developer account

This is your ticket to the Android Google Play Store.

1. Go to the Google Play Console site and sign in with a Google account (2-step verification on).
2. Choose Personal or Organization (same decision as above; Organization needs a DUNS number here too).
3. Pay the **\$25 one-time fee**.
4. Complete **identity verification**, Google will ask you to upload a valid ID. This can take from a few hours up to a couple of business days.



**Heads-up that catches everyone, the 12-tester rule.** Google now requires brand-new personal developer accounts to run a *closed test* with **12 testers for a couple of weeks** before they'll let you publish your app to the public. This isn't optional, and it's the single most common "wait, what?" moment for first-time Android launchers. It's not hard, we cover exactly how to do it in Chapter 11 (Beta Testing), but I'm flagging it *now* so it doesn't ambush you later. Start thinking about 12 friends, family, or online testers you can line up.

## Step 3: Buy your domain name

Last slow-cooker. You need a web address for your landing page and (critically) to host the privacy policy the stores require.

1. Go to any domain registrar (there are many; pick a well-known one).
2. Search for a name close to your app's name. Short and memorable beats clever.
3. Buy it, usually around **\$10-15 for the year**. You don't need any add-ons they try to upsell.

That's all for now. We'll build the actual page on it in Chapter 8.

# Building Your App, FlutterFlow + Firebase Tips and Tricks

---

This is the chapter you've been waiting for: actually building the thing. I'm not going to reproduce FlutterFlow's entire manual here, they have great docs and thousands of video tutorials, and any specific button I describe might move next month. Instead I'm going to give you the map and the tricks: the order to do things in, the mistakes I made so you can skip them, and the concepts that make everything else click.

## Build in this order (it matters)

Beginners often get stuck endlessly polishing one screen before the rest of the app even exists. Don't do that, there's a smarter order. Here's the sequence that keeps you moving, and notice you do not build the whole thing from a blank canvas:

1. Sketch your screens on paper first. Seriously, paper. Draw a box for each screen and arrows for how the user moves between them. Ten minutes here saves hours of confusion, and it's also what lets you write a good prompt in the next step and judge what's worth keeping. This is your blueprint.
2. Use FlutterFlow's AI builder to generate a base, not the finished app. This is the trick that saved me the most time, and it's where the prompt you built back in Chapter 4 finally gets used. Remember that detailed, FlutterFlow-ready prompt you had Claude write from your idea? This is the moment for it: open FlutterFlow's built-in AI builder and paste that prompt in. Instead of placing every screen and widget by hand from zero, the AI reads your prompt and generates a rough starting version, the basic screens, a layout, some look-and-feel ideas. In a few minutes you go from a blank project to something on the canvas to react to, which is far easier than staring at nothing. (If your app has evolved since Chapter 4, just pop back into your Claude project, tell Claude what changed, and ask it to update the prompt before you paste it in.)

One honest warning: the AI builder is not fantastic. Treat what it gives you as a rough first draft and nothing more, it'll get the bones roughly right and plenty of things wrong. Do not expect a working, finished app to fall out of it. Use it for the base, and that's it; you do the rest. (And to be

clear, this is not the prompt-builder trap from Chapter 3, here the result lands inside FlutterFlow, where you own it and edit it freely. It's a scaffold, not a locked platform you can't leave.)

1. Then take over, this is where the real building happens. With the AI base in front of you, you do the work. Edit the pages. Delete the screens that don't fit. Rebuild a page from scratch when the AI made a mess of it, I started several over completely, and that's totally normal. From here on you're driving: arranging widgets, fixing the layout, getting the right set of screens in place. The AI gave you a head start; the app becomes genuinely yours in this step.
2. Get every page looking exactly how you want, before you make anything work. This is the order I swear by: do all your visual design first. Go screen by screen and get the layout, fonts, colors, spacing, and images looking the way you want them, across the whole app, until you're genuinely happy with how it all looks. Don't wire up a single button yet. It's much easier to perfect the look while you're not also juggling whether things function, and it keeps you from rebuilding logic later just because you moved a screen around.
3. Once you love how it looks, then make it work. Now you wire everything up: the buttons, the navigation between screens, and the backend data (saving and loading from Firebase). My biggest recommendation here: do it section by section, and send a screenshot of each part to Claude as you go. Show Claude the screen you're working on, tell it what you want that button or section to do, and let it walk you through exactly which actions to add, or write any small bit of code you need. Working in small, screenshot-sized chunks like this is the easiest and safest way to make sure every piece actually works before you move to the next one. Don't try to wire the whole app at once; one section at a time, confirmed working, then onward.

## The concept that unlocks everything: data

Most apps need to remember things, a to-do list remembers your tasks, a login remembers who you are. This is where Firebase comes in, and where beginners get scared. Don't be. Here's the whole idea in plain terms:

- A "collection" is like a spreadsheet. For example, a collection called "tasks."
- Each "document" is a row in that spreadsheet. One task.
- Each "field" is a column. The task's title, whether it's done, who created it.

That's it. When your app "saves a task," it's adding a row to a spreadsheet in the cloud. When it "shows your tasks," it's reading rows back. Hold that picture and Firebase stops being mysterious.

## Connecting FlutterFlow to Firebase

FlutterFlow has a guided setup that walks you through linking your project to a free Firebase account. Follow their official, current walkthrough for this step exactly, it's the one place I tell you to follow the official guide click-for-click, because the screens change and getting it slightly wrong causes confusing errors later. Take it slow, and don't skip the part where it sets up "authentication" if your app has logins.

## Tips and tricks I learned the hard way

- FlutterFlow's templates exist, but don't feel chained to them. It does come with pre-built pieces like login screens and list layouts. Honestly, though, they're hit or miss, and you may well find they don't look the way you want. That's no problem at all: building your own page or component is far easier than you'd expect. Anything the AI builder didn't already make for you, you can make yourself, and you don't have to figure it out alone. While you're still learning how FlutterFlow works, your safest bet is the same method I lean on everywhere else: go step by step, screenshotting whatever you're working on to Claude as you build each piece. Do that for a little while and something clicks, you start to actually understand how FlutterFlow fits together, and building a page or a component stops feeling hard at all.

- Know how testing actually works (and its limits). FlutterFlow gives you a few ways to try your app right in the browser as you build: a quick Preview (instant, but mostly just the look, it won't run your Firebase data or logins), and Test/Run modes (slower, but they run more of the real logic). Lean on these for most of your building. Heads-up so you don't lose hours like I did: truly running the app on your own phone is finicky, it requires a paid plan plus extra setup (Xcode on a Mac for iPhone, or Android Studio for Android), and it doesn't "just work" out of the box. Don't stress about getting real-phone testing going early. The proper real-device testing happens at the beta stage with TestFlight and Google's testing track, which we cover step by step in Chapter 11. For now, the in-browser modes are enough. And here's a big reassurance while you're wiring: FlutterFlow has a built-in Project Issues checker that constantly scans your app and flags anything broken or incomplete, a half-finished action, a missing field, a typo in a piece of custom code, with a clear error that points you straight to the widget causing it. It won't even let you build or publish while those errors are unresolved. So you don't have to wire something up and just hope it's right: if a piece is genuinely broken, FlutterFlow tells you, in red, with the location. (The one thing it can't catch is when something is technically valid but you wired the wrong thing, say, a button that works fine but goes to the wrong screen, which is exactly what Test/Run mode is for.)
- Name everything clearly as you go. "Button," "Button2," "Button3" will haunt you. Name them "LoginButton," "SaveTaskButton." Your future self, debugging at midnight, will thank you.
- You don't have to save manually, but know where the safety net is. Unlike some tools, FlutterFlow autosaves as you work; there's no "save" button to remember. What confuses people coming from other builders is that the rollback option is hidden rather than missing. FlutterFlow keeps automatic snapshots of your project, and you can also save named versions, both of which you can "Peek" at and revert/restore to if a change breaks something. So the "go back to before I broke it" button does exist; it just lives in the versioning/snapshots menu rather than front and center. Find it now, before you need it. (Fair warning: some people find restore a little finicky, so for a really big, risky change it doesn't hurt to also duplicate the project as a backup first.)
- Aim for done, not perfect, but that's about features, not looks. You've already made your pages look the way you want (that was the design-first step), so this is a different warning: don't keep piling on features or chasing some imaginary flawless version before your core app actually works. Your first release just needs to do its one main job well; everything else is a "version 2" idea. And expect things to break along the way, they will, for everyone. A bug is a normal part of building, not a sign you're bad at this.

- When you're stuck, go to Claude first. This is my go-to, and it's the fastest fix nine times out of ten: screenshot the screen or the error, paste it to Claude, describe what you're trying to do, and let it walk you through the fix step by step. If, and only if, it's something Claude genuinely can't sort out, then search the exact error or situation on YouTube or the FlutterFlow community forum, where someone has almost certainly hit it before and explained it.

The "good enough" milestone. You don't need every feature. You need the one core thing from your Chapter 1 worksheet to work, reliably. You've already designed your pages to look the way you want, so this milestone is purely about function: the moment a real person could open your app and actually get that main job done, you've crossed the hardest line in this entire guide.

## Your action step: build your core screen and get it working

By the end of this chapter your goal is one concrete thing: the single most important screen of your app, doing its main job, working in Test/Run mode. Not the whole app. Just the heart of it, functioning. Once that exists, everything left is addition, not invention, and you'll know in your bones that you can finish.

Next: how your app can actually make money.

### Stuck on something in FlutterFlow?

Their official YouTube channel is a goldmine, a huge library of tutorials covering just about everything, from your very first screen to connecting Firebase and beyond. Anything you get stuck on, you can most likely find it explained here:

[youtube.com/@FlutterFlow](https://youtube.com/@FlutterFlow)

# How Your App Makes Money

---

Even if money isn't your main goal, this chapter matters, because the choice you make here affects how you build, so it's better to decide now than to bolt it on later. I'll keep it simple: there are really only a few ways apps make money, and you just need to pick the one that fits.

## The four ways apps make money

1. Free (no money). Totally valid, especially for a first app, a portfolio piece, or something you're building to learn. No payment setup, no complications. If your goal is to launch and learn, there is zero shame in free.
2. Paid up front. The user pays once to download. Simple, but a hard sell these days, people are wary of paying before they've tried something. Usually not the best choice for an unknown first app.
3. In-app purchases & subscriptions. The user downloads free, then pays inside the app, either one-time purchases (unlock a feature, remove ads) or recurring subscriptions (monthly/yearly for premium access). This is how most modern apps make money, and subscriptions are the dominant model for good reason: predictable, recurring income.
4. Ads. You show ads and get paid per view/tap. Easy to add, but it takes a lot of users to make meaningful money, and too many ads annoy people. Often best combined with a "pay to remove ads" option.

## My honest recommendation for a first app

Pick the simplest model that matches your goal:

- Just want to launch and learn? Go free. Remove all the payment complexity from your first launch.
- Want recurring income and your app delivers ongoing value? Subscriptions, via the tool below.
- Have lots of casual users and a simple app? Ads with a remove-ads option.

Resist the urge to build every money model at once. Like the app itself: one thing, done well.

## The tool that makes payments sane: RevenueCat

First, let's clear up the thing that confuses almost everyone, because it confused me too. Your users still pay through their Apple or Google account, that never changes. When someone's free trial ends and they subscribe, the charge goes through their Apple ID on an iPhone or their Google account on Android, exactly as you'd expect. Apple and Google are the cash register. They take the money, keep their cut (roughly 15-30%), and pay the rest out to you. For anything digital sold inside your app, a subscription, premium features, virtual items, you're actually required to use their payment systems; you're not allowed to slip in your own credit-card form to dodge the fee, and trying to is a fast way to get your app rejected.

So if Apple and Google handle the actual money, what's RevenueCat, and why does everyone use it? Here's the simplest way to picture it: Apple and Google are the cash register that takes the payment; RevenueCat is the manager standing behind the counter who keeps track of everything. It doesn't touch the money itself, it sits behind your app and manages the messy parts of subscriptions for you. Specifically, it:

- Lets you set up your subscription and free trial once and have it work across both stores, instead of you having to build and maintain Apple's billing system and Google's billing system separately (they're two different beasts, each with its own quirks).
- Tells your app, at any moment, "is this person currently a paying subscriber, or still in their trial?", which is how your app knows whether to unlock the paid features for them.
- Quietly handles the genuinely fiddly stuff: renewals, cancellations, refunds, failed-payment retries, and restoring someone's subscription when they get a new phone or log in on a different device.
- Gives you a clean dashboard showing trials, sign-ups, cancellations, and revenue.

Could you skip RevenueCat and wire payments straight into Apple and Google yourself? Technically yes. The reason almost nobody does is that you'd be building and maintaining all of that billing logic twice, once for each store, and that's exactly where beginners sink days and introduce bugs. RevenueCat is just a much friendlier front door to the same Apple/Google billing your users were always going to use. FlutterFlow connects to it directly, and it's free until you're earning a meaningful amount, so there's no real downside to starting with it. This is a "learn from my pain" recommendation: if you're doing subscriptions or in-app purchases, set up RevenueCat from day one.



## Your action step: pick your model and write it down

Add one line to your project notes:

"My app will make money by: \_\_. For my first version, I will set up: \_\_\_\_ (or 'nothing, launching free')."

If you picked free, you're done with this chapter and you've saved yourself real complexity, enjoy that.

If you picked subscriptions or purchases, here's the important timing note: don't try to fully set up RevenueCat right now. Most of the work, connecting your actual subscription products, can't be done until your developer accounts and store listings exist, which happens later (around submission, Chapter 12). If you sit down to wire it up today, you'll hit a wall because there are no products to connect to yet. So for now, just know that RevenueCat is the path you'll take. When you're ready to set up payments, once your store listings exist, follow RevenueCat's own FlutterFlow guide. I'm pointing you to their guide rather than printing the steps here on purpose: payment setup spans three platforms (Apple, Google, RevenueCat) that all redesign their screens often, so their live guide will always be more accurate than any screenshot I could freeze in a guide. And as always, if a step trips you up, screenshot it to Claude and work through it one piece at a time.

Next, we step out of the app itself and build the simple website you need, both to promote your app and to host the legal page the stores require.

# Building a Simple Landing Page (with Free Hosting)

---

You need a website. Not a big one, a single, simple page. It does two jobs: it gives people a place to learn about your app (and a link you can share), and just as importantly, **it's where you'll host the privacy policy that Apple and Google require before they'll approve your app.** That second job is non-negotiable, which is why this chapter comes before submission.

The good news: you can do this for free, and you don't need to be a web developer. I'll show you the simple path.

## What your landing page actually needs

Keep it to one page with these pieces:

- **Your app's name and a one-line description** (the one-sentence pitch from Chapter 1).
- **A few sentences on what it does and who it's for.**
- **A screenshot or two** of the app.
- **Download buttons / links** to the App Store and Google Play (you'll add the real links after you launch).
- **Links to your Privacy Policy and Terms** (we create those in the next chapter, they'll live on this site).
- **A way to contact you** (an email address is plenty).

That's a complete, professional landing page. Don't overbuild it.

## The simplest way to build it (no coding required)

You've got two honest options, and I'll give you both:

**Option A - A no-code site builder (easiest).** Many domain registrars and website services let you build a simple page by dragging blocks around, then publish it to your domain. If "writing any HTML at all" sounds stressful, start here. It's the fastest route to a live page.

**Option B - A simple HTML page with free hosting (this is the route I took).** A landing page is just one file of HTML, the basic language of web pages. And here's the part that makes this genuinely easy: you don't have to write that HTML yourself. I had Claude write mine for me. I described the page I wanted, my app's name, a tagline, a screenshot or two, the store download buttons, and links to my privacy policy and terms, and Claude produced the whole HTML file. Then I hosted it for free on GitHub, using a feature called GitHub Pages, and honestly it was simple to set up. Below is the rough shape of a basic page just so it's not a total mystery, but you can absolutely have Claude generate yours instead of typing it:

```
<!DOCTYPE html>
<html>
<head>
  <title>My App Name</title>
</head>
<body>
  <h1>My App Name</h1>
  <p>The one-line description of what my app does.</p>
  
  <a href="APP_STORE_LINK">Download on the App Store</a>
  <a href="PLAY_STORE_LINK">Get it on Google Play</a>
  <a href="privacy.html">Privacy Policy</a>
  <a href="terms.html">Terms of Service</a>
  <p>Contact: youremail@example.com</p>
</body>
</html>
```

Don't worry about understanding every line, the point is just to see that a web page is simpler than it looks. In practice you won't hand-type this: describe what you want to Claude, let it write the HTML, then ask it to tweak anything you'd like changed. (If you'd rather start from a ready-made design, you can also grab a free landing-page template, search "free HTML landing page template", and have Claude adapt it to your app.)

## Free hosting (so the page is actually live on the internet)

A page sitting on your computer isn't visible to the world, it needs hosting. The free option I used is GitHub Pages: you put your page file into a free GitHub account, switch on the Pages feature, and GitHub serves your site to the world at no cost, and it lets you connect the domain you bought in Chapter 5. It sounds technical, but it's a well-trodden, beginner-friendly path, and Claude can walk you through each step (or search "GitHub Pages custom domain setup" for guides). If GitHub isn't for you, there are other free static-site hosts, search "free static site hosting." Whichever you choose, the flow is the same: upload your page file(s) → connect your domain → your page is live at your web address.

**Why this page is doing double duty:** when you submit your app, the stores will ask for a publicly accessible Privacy Policy URL. That URL will be a page on this site. So even if you don't care about a fancy landing page, you need this site to exist to get approved. Build it now and the submission chapter goes smoothly.

## Your action step: get a live page at your domain

You don't need the store links yet (you don't have them until you launch, you'll add them after). Your goal for this chapter:

- ☒ A simple one-page site is built (no-code builder or HTML template)
- ☒ It's hosted for free and connected to your domain
- ☒ You can visit your web address and see your page
- ☒ You've left obvious spots to add Privacy Policy and Terms links (next chapter)

Once you can type your domain into a browser and see your own page, you're ready for the legal pages, which is exactly where we go next.

# Privacy Policy and Terms of Service (the Painless Version)

---

The word "legal" makes people freeze. Don't freeze. For a simple first app, this chapter is genuinely quick, and you do not need to hire a lawyer to get started. You need two documents, you can generate them for free, and they live on the website you just built. Let's knock it out.

## The two documents you need

1. Privacy Policy (required, non-negotiable). This explains what information your app collects and what you do with it. Both Apple and Google require a publicly accessible privacy policy URL before they'll approve your app. There is no skipping this one. Even if your app collects almost nothing, you still need a policy that says so.
2. Terms of Service / Terms & Conditions (strongly recommended). This sets the rules for using your app and protects you, limiting your liability, explaining what users can and can't do. Not always strictly required, but you want it.

## How to create them for free

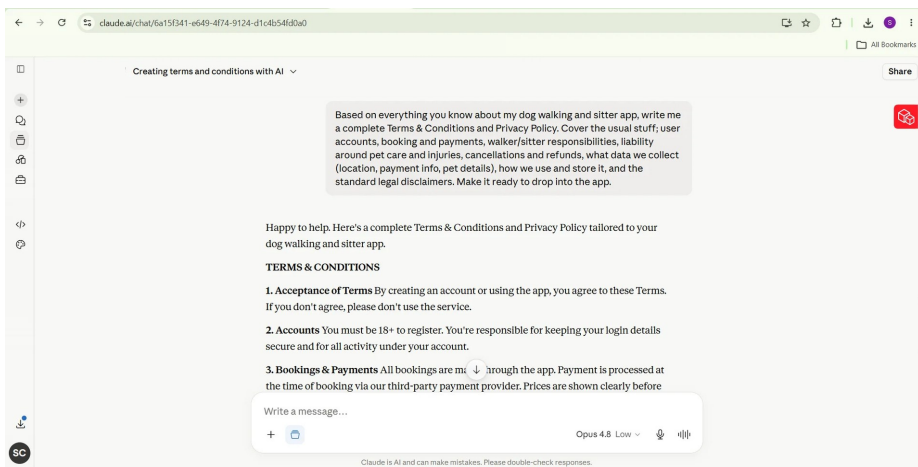
You do not write these from scratch, and you don't need a lawyer for a simple app. You've got two easy, free routes:

Have Claude write them (this is what I did). Because you've been building inside your app's Claude project (Chapter 4), Claude already knows your app, what it does, that it has logins, that it uses Firebase, whether it shows ads or has subscriptions. So you can simply ask Claude to write a privacy policy and a terms of service for your app, and it'll draft both from what it already knows. Give them a read to make sure everything your app collects is covered, ask Claude to adjust anything that's off, and you're done. Quick and painless.

Or use a free generator. There are several reputable free privacy policy generators and terms of service generators online (search those exact terms). They ask you simple questions about your app, does it collect emails? use analytics? show ads?, and produce a policy from your answers.

Either route works, and one quick caveat for both: Claude (like the generators) isn't a lawyer, so make sure the result accurately matches your app, and if you're doing something heavier than a simple app, handling payment details directly, health data, anything aimed at children, or operating where privacy laws are strict, it's worth having a professional look it over. For a straightforward first app, though, either route above is genuinely fine.

Whichever you choose, the important thing is the same: you need to know what your app actually collects, so you can confirm the policy covers it (or answer the generator's questions) accurately. Think through: - Do users create accounts / log in? (Then you collect emails, names, etc.) - Do you use Firebase? (Then you store user data, make sure that's covered.) - Do you use any analytics or ads? (Each needs disclosing.) - Do you make any in-app purchases or subscriptions? (Disclose it.)



Be honest and thorough here, under-disclosing what you collect is one of the things that gets apps rejected, and it's so easily avoided.

## Putting them online

However you made them, Claude or a generator, you'll have the text for each. Put each one on your website as its own page (the same site you set up in Chapter 8), at addresses like *yourdomain.com/privacy* and *yourdomain.com/terms*, and link to them from your landing page. If you went the Claude + GitHub Pages route from Chapter 8, this is easy: have Claude turn each policy into its own simple HTML page, drop them in alongside your landing page, and they'll be live at those addresses. Now you've got two real URLs to hand to the app stores. That's the entire deliverable.

**A note on honesty, not just compliance:** your privacy policy is a promise to your users. Whatever it says you do with their data, actually do that. The point isn't to copy-paste your way past the store reviewers; it's to be straight with the people using your app. Collect as little as you need, say so plainly, and you'll rarely have problems.

## Your action step: create, post, and save the URLs

- ☒ Listed what data my app collects (logins, Firebase, analytics, ads, purchases)
- ☒ Created a Privacy Policy (had Claude write it, or used a free generator)
- ☒ Created Terms of Service the same way
- ☒ Read both to confirm they match what my app actually does
- ☒ Posted both on my website as their own pages
- ☒ Saved both URLs somewhere handy, I'll paste them into the store listings soon

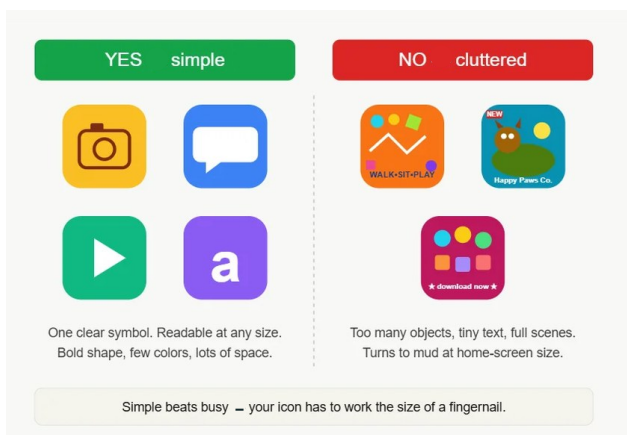
With your legal pages live, the unglamorous prep is basically done. Now we get to the fun, creative part: making your app look irresistible.

# Creating Great Visuals to Promote Your App

Here's something that surprised me: you can't even submit your app without certain visuals ready. The stores require an icon and screenshots as part of your listing. So this isn't a victory-lap "make it pretty" chapter, it's required prep for submission, which is why it comes right before it. The bonus is that good visuals are also what make people actually tap "download." Let's make yours look like they belong.

## What you actually need

1. An app icon. The little square that lives on the phone's home screen. This is your app's face. It must be simple, recognizable at tiny sizes, and ideally one clear symbol, not a detailed picture, not lots of text. Look at the apps on your own phone: the memorable icons are almost all one bold, simple shape or letter.



2. Screenshots for the store listing. These are the images people swipe through on your app's store page before downloading. Both stores require several, at specific sizes. These are your sales pitch, most people decide from the screenshots alone.

3. (Optional) A feature graphic / promo banner. Google Play asks for a feature graphic (a wide banner). Nice to have, not hard to make.

## How to make them for free (and have them look good)

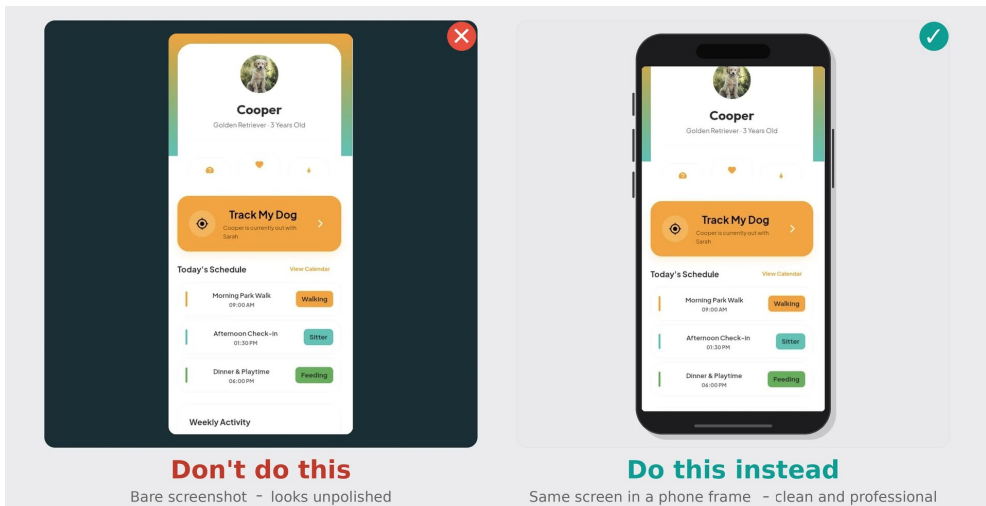
You do not need to be a designer or buy software. A free design tool with app-store templates (search "free app icon maker" and "app store screenshot template") will give you correctly-



sized canvases so you don't have to guess dimensions, getting the sizes wrong is the #1 visual headache, and templates solve it for you.

For your icon: - One simple idea: a single symbol, or your app's first letter, on a solid or simple background. - Bold colors, high contrast, readable at thumbnail size. - Test it small: shrink it down and see if it's still clear. If you can't tell what it is at tiny size, simplify.

For your screenshots, the trick that makes them look professional: don't just use raw phone screenshots. Put each screenshot inside a phone "frame" (templates do this for you, you can search for "mock-ups") and add a short caption above or below each one describing the benefit: "Track your habits in seconds," "Never forget a task again." Captioned, framed screenshots look ten times more professional than bare ones, and they sell the app instead of just showing it.



**Make your first screenshot your best one.** Most people only glance at the first one or two. Lead with the screenshot that shows your app's single best moment, the core value from your Chapter 1 worksheet, with the clearest caption. Don't bury your best shot in position four.

## A word on consistency

Pick two or three colors and one font, and use them across your icon, screenshots, and landing page. This consistency is the difference between "looks like a real product" and "looks like a first attempt." It costs nothing and matters a lot.

## Your action step: build your visual asset pack

Create a folder and fill it:

- App icon (simple, bold, readable when tiny)
- 3-5 screenshots, framed in phone mockups, each with a benefit caption
- First screenshot = your single best/clearest one
- (For Google) a feature graphic banner
- A consistent color + font theme across everything

With your visuals in hand, you now have everything the stores will ask for. Before we submit, though, there's one required step left, testing your app with real people, and for Android, it's mandatory. That's next.

# Beta Testing Before You Launch

---

You are so close to launching. Please don't skip this chapter to get there faster, testing is what stands between you and a flood of 1-star reviews from real users hitting bugs you never saw. And for Android, it's not even optional: Google now requires it before you can publish. Let's do it right.

## Why testing matters (the unglamorous truth)

You have been staring at your own app for weeks. You know exactly which buttons to tap and in what order, so you never hit the weird paths. Real people don't know any of that. They'll tap things you never expected, on phones different from yours, and find problems you're blind to. Better they find them now, quietly, than in public reviews after launch.

## Apple: TestFlight

Apple gives you a free tool called TestFlight for sending your app to testers before the public launch.

- You upload your app, then invite testers by email or a shareable link.
- Testers install TestFlight, then your app, and use it like normal.
- They can report bugs; you fix and send updated versions.

It's a smooth process and a great safety net. Aim for at least a handful of testers who aren't you, friends, family, anyone with an iPhone willing to poke at it for ten minutes.

## Google: closed testing (this one's mandatory, remember the 12-tester rule)

Back in Chapter 5, I flagged this so it wouldn't ambush you. Here it is in full:

New Google Play personal developer accounts must run a "closed test" with around 12 testers for roughly two weeks before they're allowed to publish the app publicly. This is a real requirement, not a suggestion, and it's the most common thing that blindsides first-time Android launchers. If you didn't start lining up testers in Chapter 5, do it now.

A note if you signed up as an organization: don't assume this doesn't apply to you. Some organization accounts have been exempt from the 12-tester requirement, but Google has been steadily expanding it, so it's increasingly likely to hit organization accounts too. Rather than bank on skipping it, plan as if you'll need your ~12 testers either way; if it turns out your account is exempt, you've lost nothing. Check the requirement in your own Play Console when you get there, since it's the source of truth for your account.

How it works: - In the Play Console, you set up a "closed testing" track. - You add your ~12 testers (by email, or via a Google Group). - They opt in and actually use the app over the testing period. - Once you've met the requirement, Google unlocks the ability to apply for production (public) release.

**Where to find 12 testers if you don't have them:** friends and family are the obvious start. Beyond that, there are online communities specifically for app makers who test each other's apps to satisfy this exact requirement, search "Google Play 12 testers" and you'll find groups where people swap testing. It's a normal, well-trodden path. Don't let this requirement scare you; thousands of solo developers clear it every week.

## What to ask your testers

Don't just say "try my app." Give them a tiny mission so you get useful feedback: - "Can you sign up and do [the main thing] without getting confused?" - "Did anything break, freeze, or look wrong?" - "Was anything unclear?"

Watch for the spots where multiple testers stumble, those are real problems worth fixing before launch. One person's nitpick you can note for later; three people hitting the same wall is a priority.

## Your action step: get your app into testers' hands

- ☒ (Apple) Uploaded to TestFlight and invited testers
- ☒ (Google) Set up closed testing and added ~12 testers
- ☒ Gave testers a clear mission, not just "try it"
- ☒ Collected feedback and fixed anything multiple people hit
- ☒ Started the clock on Google's testing period (it takes ~2 weeks, so don't wait)

Once your testers can use your app without hitting walls, you're ready for the moment this whole guide has been building toward: submitting to the stores.

# Submitting to the App Store and Google Play

---

This is it, the submit button. It feels huge and scary, and it's really just carefully filling out a long form and uploading what you've already made. You've done all the prep; now we assemble it. I'll also tell you the common reasons first apps get rejected, so you sail through on the first try instead of bouncing back and forth for weeks like I did.

## Before you hit submit: the universal checklist

Have these ready (you made them all in earlier chapters):

- ☐ Your finished, tested app
- ☐ App icon and screenshots (Chapter 10)
- ☐ Privacy Policy URL and Terms URL (Chapter 9)
- ☐ An app name, description, and category
- ☐ A support contact (email)
- ☐ If your app has a login: a working test account (username + password) to give the reviewers

That last one is quietly one of the most important things in this chapter, see the rejection reasons below.

## Submitting to Apple

The pieces happen across Apple's tools (App Store Connect, where you fill in your listing, and an upload from FlutterFlow/your build):

1. Create your app's listing in App Store Connect.
2. Fill in the name, description, category, screenshots, icon, and your Privacy Policy URL.
3. Get your app build to Apple. **The easy way, and the reason you don't need a Mac, is FlutterFlow's built-in "Deploy to App Store"** (on a paid plan): FlutterFlow builds your app and sends it to Apple *from the cloud*, and you'll get an email from App Store Connect once the build arrives. No Mac, no Xcode, no terminal commands.
4. Fill in the **App Privacy** section, Apple asks you to declare what data you collect. Answer it honestly and match your privacy policy.
5. Provide the demo login if your app needs one.
6. Submit for review.

**Wait, don't I need a Mac to put an app on the App Store?** This trips up a lot of beginners, and I learned it the hard, expensive way. Back when I built my first app on one of those prompt-based AI builders, the kind that has to be wrapped by Median.co to be publishable (Chapter 3), that whole setup forced me to buy a MacBook just to get my file uploaded to Apple. Another \$500 gone, on top of everything else those tools had already cost me. Here's the thing I wish I'd known: with FlutterFlow you *don't* need a Mac. You only need one if you do it *the manual way*, exporting your code and uploading it yourself with Apple's tools, which only run on macOS. But FlutterFlow's one-click "Deploy to App Store" does the whole build-and-submit in the cloud for you, so for a normal app you can publish to the iPhone App Store from a Windows PC, a Chromebook, anything. **Bottom line: don't buy a Mac just to publish. Use the one-click deploy.**

**And to be crystal clear about "custom code," because the wording confuses people:** the custom functions and actions you write *inside* FlutterFlow, little snippets of Dart for logic, calculations, formatting, points, greetings, and so on, are **totally fine with no Mac**. FlutterFlow compiles them in the cloud along with everything else. You can write dozens of them (most real apps do) and still publish from any computer. The *only* rare exception is true "custom **native** code", dropping in a specialized iOS plugin or native package that has to be configured outside FlutterFlow in Xcode (think niche hardware integrations or a third-party SDK that ships its own native framework). A normal first app never goes near that. So: your own custom functions = no Mac; obscure native plugins = the one scenario where a Mac comes up.

**Apple's review** is done by actual humans and usually takes anywhere from a day to a few days. They're stricter than Google.

## Submitting to Google Play

1. Create your app in the Play Console and fill in the **store listing** (name, descriptions, screenshots, icon, feature graphic).
2. Complete the required questionnaires: content rating, target audience, data safety (Google's version of the privacy declaration), and a few policy questions.
3. Add your Privacy Policy URL.
4. Upload your app, then promote it from closed testing (Chapter 11) toward production.
5. Submit for review.

**Google's review** is often faster than Apple's, but remember the closed-testing requirement has to be satisfied first.

## The reasons first apps get rejected (and how to dodge each)

Learn from the mistakes that cost everyone time. Most rejections come from this short list:

- **Broken or missing demo login.** Reviewers can't get into your app, so they reject it. *Fix:* always provide a working test account, and double-check it works right before submitting.
- **A crash or bug the reviewer hits.** *Fix:* this is what Chapter 11 testing was for. Test the main paths on a real device one more time.
- **Privacy declarations that don't match reality.** You said you collect nothing, but you have logins and analytics. *Fix:* make your in-store privacy answers honestly match your actual app and your privacy policy.
- **Looks too thin / "spam."** Apple in particular rejects apps that feel like an empty shell or a repackaged website with little value. *Fix:* make sure your app genuinely *does its one core thing well*. (Your Chapter 1 worksheet was protecting you from this all along.)
- **Placeholder content or broken links.** Leftover "Lorem ipsum," a privacy link that 404s. *Fix:* click every link in your listing before submitting.
- **Missing or wrong-sized screenshots/icon.** *Fix:* the templates from Chapter 10 keep you safe here.



If you do get rejected, don't spiral. It is normal, especially with Apple, and it is not a verdict on you. They send a specific reason. Read it calmly, fix that one thing, and resubmit. Most rejections are a five-minute fix and a resubmit, not a rebuild. I got rejected on my first submission and approved on the next, it happens to almost everyone.

## Your action step: submit

- ☒ Ran through the universal checklist above
- ☒ Triple-checked the demo login works (if applicable)
- ☒ Clicked every link in the listing
- ☒ Privacy declarations match the real app
- ☒ Submitted to Apple, Google, or both
- ☒ Took a breath, you just submitted an app you built from nothing

Whatever happens next, approval or a quick fix-and-resubmit, you've done the thing most people only talk about. The final chapter is about what comes after the "approved" email.

# You're Live. Now What? (Post-Launch)

The "your app is now available" email is one of the best feelings there is. Enjoy it, you earned it. Then, when you're ready, this chapter covers what to do next so your app doesn't just launch and vanish. None of this is urgent on day one. Think of it as your first-30-days playbook.

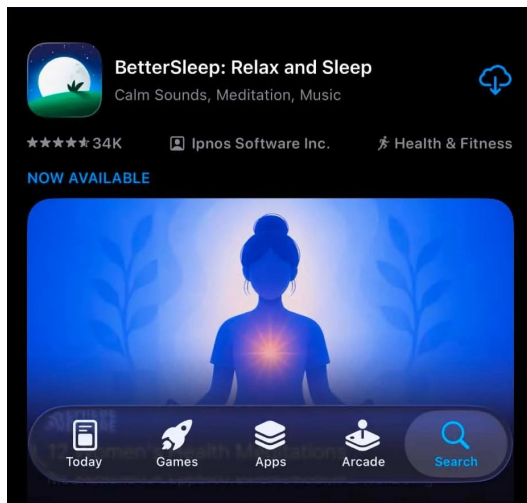
## 1. Tell people (it won't market itself)

An app in the store that nobody knows about gets zero downloads. That's not failure, it's just physics. Your first downloads come from you telling people: - Share your landing page link everywhere you have a presence. - Post in the communities where your specific users already hang out (remember, you found them back in Chapter 1's research). - Tell friends, family, and anyone who fits your audience. Early downloads and reviews matter a lot.

You don't need an ad budget to get your first hundred users. You need to show up where your people already are.

## 2. App Store Optimization (ASO), free visibility

ASO is just "making your app easier to find in the stores." The basics, which cost nothing: - Put the words people would search for into your app's title and description. - Use all your screenshot slots, with clear benefit captions. - Encourage happy users to leave reviews, rating and review volume affects how the stores rank you.



## 3. Respond to reviews

Both stores let you reply to user reviews. Do it, especially early: - Thank the positive ones briefly. - Respond to negative ones calmly and helpfully, often you can turn a 1-star into a 4-star

just by fixing their issue and replying. And everyone else reading your reviews sees that you care.

Reviews are also your richest source of "what to build next." Your users will tell you exactly what they want.

## 4. Watch your analytics (lightly)

Both stores show you basic numbers, downloads, where they come from, crashes. Firebase can show you how people actually use the app. Don't obsess, but do glance at: - Are people coming back, or installing once and leaving? - Is anything crashing? - Which screen do people drop off on?

These point you to what to improve. Let real user behavior guide you, not guesses.

## 5. Ship updates

Your first version is a starting point, not a final answer. Push updates that fix bugs and add the most-requested features (from your reviews). A steadily improving app signals life to both users and the stores. You don't need a big roadmap, just keep responding to what real users tell you.

## 6. Paid ads (optional, a tool whenever you want it)

Everything above is free, and free is a great place to begin. But when you want to reach people beyond your own circle, paid ads are how you do it, so here's the lay of the land.

For apps, the channels most people reach for are:

- Meta ads (Facebook & Instagram), you put your app in front of users based on their interests, age, location, and behavior. Great for visual apps and for finding a specific audience.
- TikTok ads, short-video ads in the feed; strong for apps with a younger audience or anything that demos well in a quick clip.
- Apple Search Ads, your app shows at the top of App Store search results when people search relevant terms.
- Google App Campaigns, install-focused ads spread across Google Search, YouTube, and the Play Store.

On every one of these you set your own budget, so you're always in control of what you spend, and you can start as small as you like.

A few suggestions to get more out of your money (take them or leave them, it's your app and your call):

- It often pays to make sure your app and listing are solid first. If people who find you organically are downloading and sticking around, ads tend to go a lot further. If you'd rather test ads early to learn, that's completely fine too, just go in knowing it's as much about learning as about installs.
- Starting small is usually the smart move. A few dollars a day on one platform is enough to get a feel for whether ads work for your app before you scale up.
- The number worth watching is cost per install, roughly, what you're paying to get one new user, and whether those users stick around. If it's costing more than a user is worth to you, that's your signal to tweak or pause. If it's working, that's your signal to lean in.
- Try one channel at a time. It's much easier to tell what's actually working when you're not running Meta, TikTok, and Google all at once.

Bottom line: ads are a real, powerful tool, and which (if any) you use is entirely up to you. Some apps grow purely on word of mouth and never spend a cent; others find their audience fastest through Meta or TikTok. Do what fits your app, your budget, and your goals.

The mindset that matters now: launching was the hard part, and you did it. From here it's a loop, listen, improve, tell more people, repeat. Apps that succeed are rarely the ones that launched perfect; they're the ones whose makers kept showing up. You've already proven you can do the hard thing. The rest is just continuing.

## Your action step: your first-30-days plan

☐ Week 1: Share your app everywhere your audience is; ask early users for honest reviews

☐ Week 1-2: Reply to every review, good and bad

☐ Week 2: Sharpen your store listing keywords (ASO) based on what you've learned

☐ Week 3: Look at your analytics, find the one biggest drop-off or bug

☐ Week 4: Ship one update that fixes or improves that one thing

☐ Week 5: Ongoing: Repeat the loop. Listen, improve, share.

☐ Optional, whenever you're ready: try a small paid-ads test on one channel (Meta, TikTok, Apple Search Ads, or Google) and see how it goes.

# A final word

When you started this guide, you already knew building an app was possible, in 2026, everyone knows there are tools for this now. That was never the hard part. The hard part was knowing where to start: cutting through the thousands of conflicting recommendations, the tutorials that contradict each other, the tools that promise the world, and figuring out which path is actually the right one for you. That noise is what wastes people's time and money, not a lack of ability. Now you've got the truth: one clear, proven path, made of doable steps, most of which cost very little, none of which require you to be a genius. You just needed someone to show you the way through, without the detours and dead ends.

That was the whole point of this guide, to be the roadmap I didn't have when I started, so you don't lose the time and money I did.

**When in doubt, Claude can most likely figure it out.** I cannot stress this enough: lean on the screenshot-to-Claude habit to help you navigate what to do and where to go next. It is a genuine lifesaver. Just be patient, stay calm through the process, and rely on Claude, your trusted AI assistant, to walk you through the whole thing. (Just remember what we covered in Chapter 4: stay engaged, use your own logic, and push back when something doesn't add up, lean on Claude, don't switch your brain off.)

With this guide, you've got a clear picture of what to do and the steps to take, without burning countless hours researching to figure it all out, or spending hundreds and hundreds of dollars only to discover that path wasn't the one you should have taken. I wasted that money and that time so you don't have to. All you have to do is follow this guide, use Claude to guide you, and the whole thing becomes as easy as 1, 2, 3.

So that's it. You've got the path, you've got the tools, and you've got Claude in your corner the whole way. The only thing left is to start.

Now go build the thing.